



UNIVERSITÄT ZU LÜBECK
INSTITUTE FOR IT SECURITY

Side-Channel Leakage Analysis with Microwalk

Thomas Eisenbarth, Jan Wichelmann

OPTIMIST 2025

CPU Side-Channel Leakage: Example

```
char caesarLookup[][26] = {
    "abcdefghijklmnopqrstuvwxyz",
    "bcdefghijklmnopqrstuvwxyz",
    "cdefghijklmnopqrstuvwxyz",
    "defghijklmnopqrstuvwxyz",
    // ...
};

void caesarEncrypt(char input[], int length, char output[], int shift)
{
    for(int i = 0; i < length; ++i)
    {
        // ... ignore non-letters ...

        output[i] = caesarLookup[shift][input[i] - 'a'];
    }
}
```

- Side-channel leakage = secret-dependent operations

- Side-channel leakage = secret-dependent operations
- Solution: Secret-independent *constant-time* code

Countering Side-Channel Attacks

- Side-channel leakage = secret-dependent operations
- Solution: Secret-independent *constant-time* code
 - Always follow the same execution path

- Side-channel leakage = secret-dependent operations
- Solution: Secret-independent *constant-time* code
 - Always follow the same execution path
 - Always access the same memory locations

Finding Side-Channel Leakages: State-of-the-Art

Tool	Approach	Level	Security	Properties			Capabilities			Output		
				Sound	Compl	Scal	PubOut	IntSec	Blind	Loc	Quan	Witn
ct-verif	reduction; product program	IR	constant-time									
SideTrail	reduction; self-composition	IR	time-invar.									
Blazer	reduction; decomposition	source	time-invar.									
Volpano et al.	type system	-	const. ctrl.									
Agat	type system	-	time-invar.									
VirtualCert	type system	cert	constant-time									
FlowTracker	type system	IR	constant-time							#		
CacheAudit	abstract interpretation; cache model	binary	cache-invar.									
CacheAudit2	abstract interpretation; cache model	binary	constant-time									
Blazy et al.	abstract interpretation; static tainting	cert	constant-time									
STAnalyzer	abstract interpretation; static tainting	source	constant-time							#		
CacheS	abstract interpretation; static tainting	lifted IR	constant-time							#		
CANAL	symbolic execution; self-comp., cache model	IR	time-invar.									
CacheFix	symbolic execution; self-comp., cache model	source	cache-invar.							#		
CoCo-Channel	symbolic execution; decomposition	source	time-invar.									
CaSym	symbolic execution; self-comp., cache model	IR	cache-invar.							#		
PitchFork	symbolic execution; static tainting	IR	constant-time							#		
Binsec/Rel	symbolic execution; self-comp., relational SE	lifted IR	constant-time							#		
ENCIDER	symbolic execution; self-comp., static tainting	IR	constant-time							#		
Pasareanu et al.	symbolic execution; model counting	IR	time-invar.									
ct-grind	taint tracking; tainted data-dependent accesses	binary	constant-time							#		
Irazoqui et al.	taint tracking; real cache attack traces	IR	cache-invar.							#		
Zankl et al.	statistical test; modexp bit correlation	binary	time-invar.									
dudetect	statistical test; resource usage measurement	binary	time-invar.									
STACCO	trace diffing; pairwise, TLS-only, basic block traces	binary	const. ctrl.							#		
DATA	trace diffing; pairwise, stat. test, leakage model	binary	constant-time							#		
CaType	refinement types; cache variance	binary	cache-invar.							#		
CacheQL	trace diffing; neural networks, game theory	binary	constant-time							#		
ct-fuzz	fuzzing; self-composition	binary	constant-time							#		
DiffFuzz	fuzzing; self-composition	IR	constant-time							#		
CacheD	dyn. symbolic execution cache variance	binary	cache-invar.							#		
Abacus	dyn. symbolic execution cache variance, sampling	binary	cache-invar.							#		
Microwalk	trace diffing; N-way, linear time	binary	constant-time							#		

(from: Jan Wichelmann: Software Defenses against CPU Side-Channels, 2024.)

- Cross-platform leakage detection framework
- Based on DBI and trace comparison
- Focus on *practical* use
- Efficient localization and quantification of leakages



¹Wichelmann et al., ACSAC 2018

How to Find Side-Channel Leakages Efficiently?

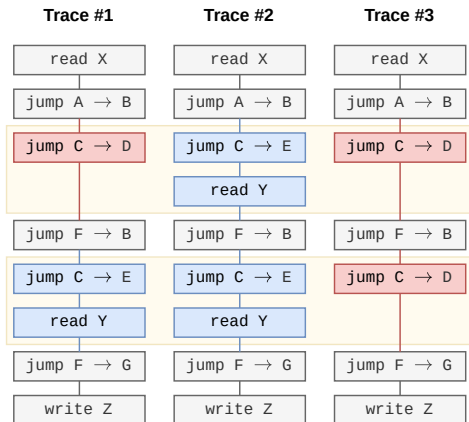
- Collect execution traces

How to Find Side-Channel Leakages Efficiently?

- Collect execution traces
- Look for trace differences $\Rightarrow$ leakages

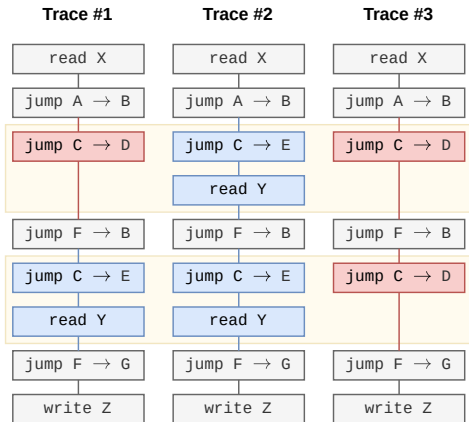
How to Find Side-Channel Leakages Efficiently?

- Collect execution traces
- Look for trace differences $\Rightarrow$ leakages



How to Find Side-Channel Leakages Efficiently?

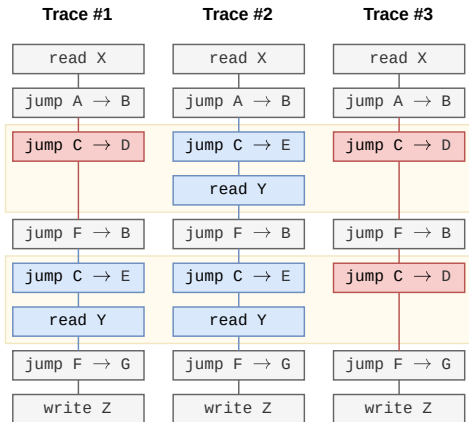
- Collect execution traces
- Look for trace differences $\Rightarrow$ leakages



- Diffing traces is **quadratic** in trace length and count

How to Find Side-Channel Leakages Efficiently?

- Collect execution traces
- Look for trace differences $\Rightarrow$ leakages



- Diffing traces is **quadratic** in trace length and count
- Are there more efficient algorithms?

- Merge traces into single tree

²Wichelmann et al., *ACM CCS* 2022

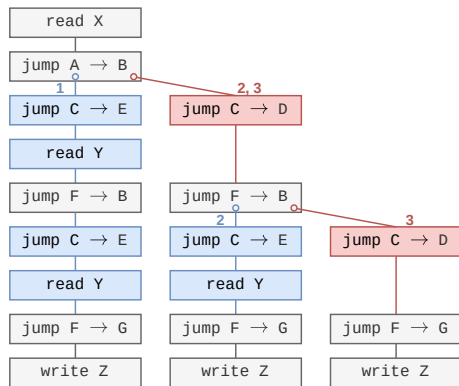
Trace Trees²

- Merge traces into single tree
- Differences produce branches

²Wichelmann et al., *ACM CCS* 2022

Trace Trees²

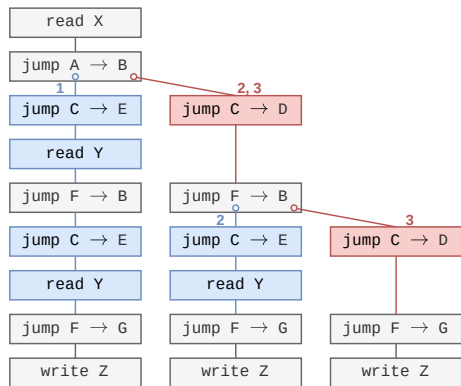
- Merge traces into single tree
- Differences produce branches



²Wichelmann et al., ACM CCS 2022

Trace Trees²

- Merge traces into single tree
- Differences produce branches

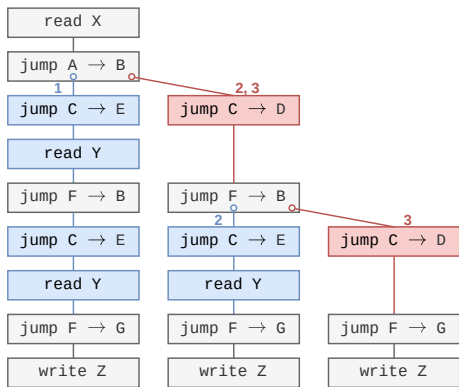


- Linear in trace length and count

²Wichelmann et al., ACM CCS 2022

Trace Trees²

- Merge traces into single tree
- Differences produce branches

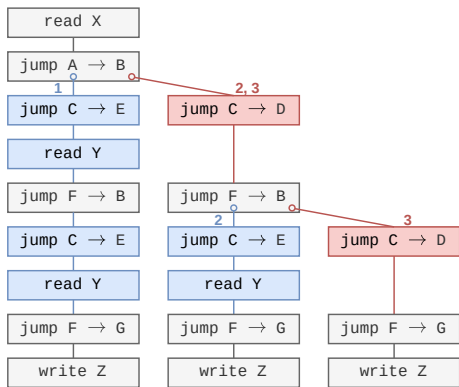


- **Linear** in trace length and count
- Leakages encoded in data structure
⇒ fast analysis

²Wichelmann et al., ACM CCS 2022

Trace Trees²

- Merge traces into single tree
- Differences produce branches

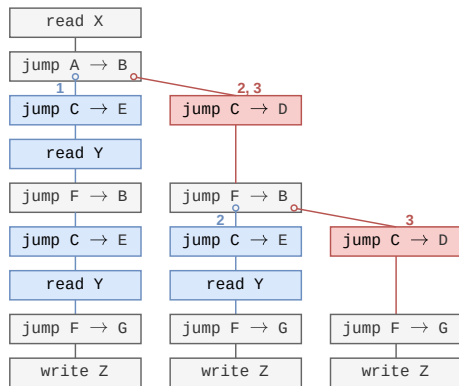


- **Linear** in trace length and count
- Leakages encoded in data structure
⇒ fast analysis
- Tends to *saturate*, i.e., hide later leakages

²Wichelmann et al., ACM CCS 2022

Trace Trees²

- Merge traces into single tree
- Differences produce branches



- **Linear** in trace length and count
- Leakages encoded in data structure
⇒ fast analysis
- Tends to *saturate*, i.e., hide later leakages
- Randomness particularly susceptible to saturation

²Wichelmann et al., ACM CCS 2022

Experimental Idea: Trace Graphs

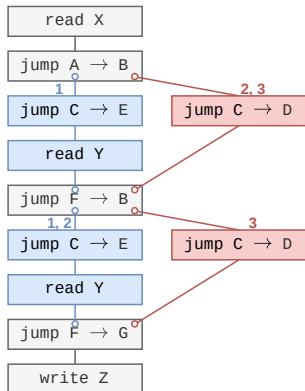
- Use LLVM plugin to put *merge markers* into code

Experimental Idea: Trace Graphs

- Use LLVM plugin to put *merge markers* into code
- Same algorithm as trace tree, but re-merge branches at markers

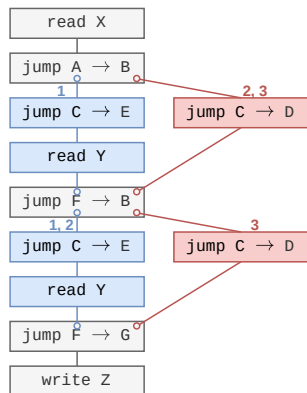
Experimental Idea: Trace Graphs

- Use LLVM plugin to put *merge markers* into code
- Same algorithm as trace tree, but re-merge branches at markers



Experimental Idea: Trace Graphs

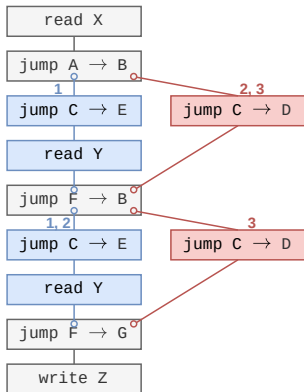
- Use LLVM plugin to put *merge markers* into code
- Same algorithm as trace tree, but re-merge branches at markers



- Very efficient

Experimental Idea: Trace Graphs

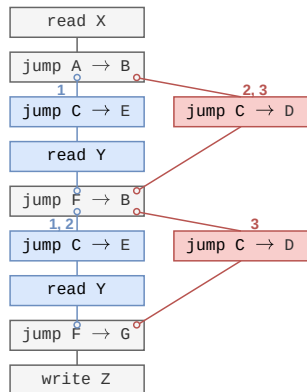
- Use LLVM plugin to put *merge markers* into code
- Same algorithm as trace tree, but re-merge branches at markers



- Very efficient
- Preserves program control structures

Experimental Idea: Trace Graphs

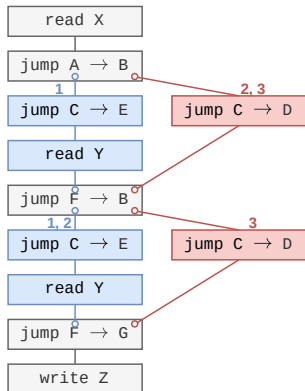
- Use LLVM plugin to put *merge markers* into code
- Same algorithm as trace tree, but re-merge branches at markers



- Very efficient
- Preserves program control structures
- ⇒ Isolation of randomized regions

Experimental Idea: Trace Graphs

- Use LLVM plugin to put *merge markers* into code
- Same algorithm as trace tree, but re-merge branches at markers



- Very efficient
- Preserves program control structures
- ⇒ Isolation of randomized regions
- Promises precise statistical analysis

Summary: Microwalk

- Ongoing project since 2018
- Leakage detection through dynamic analysis
- Focus on performance, accuracy and usability
- Ongoing work:
 - Generic support of randomized implementations
 - Improved analysis templates
 - Rendering of analysis results and coverage
 - Inclusion in real-world development workflows

